

TinyML-based Neurological Tremor Detection using ESP32

Shubham Rane
23BEC0235

Mohammad Zayd Khan
23BVD0026

Suryansh Rai
23BEC0107

shubhamsatish.rane2023@vitstudent.ac.in mohammadzayd.khan2023@vitstudent.ac.in suryansh.raii2023a@vitstudent.ac.in

Abstract—Tremors of the nervous system are typically associated with many types of diseases like Parkinson’s and require ongoing evaluation for advance signs. In this paper, we describe a real time tremor detection device based on TinyML. The device uses an ESP32 microcontroller, an MPU6050 IMU sensor, and a MAX30102 pulse oximeter sensor to detect and monitor tremors in real time. The MPU6050 sensor provides measurements of motion on three different axes which can be used to determine patterns in motion that may indicate tremors. The MAX30102 provides additional heart rate data that can be combined with the IMU data as another measure of overall condition. To provide on-device processing and inference, a small lightweight neural network was created using Python and converted to TensorFlow Lite format to run on the ESP32 microcontroller. The system is designed to perform inference locally, without the need for cloud processing. Thus, this low-cost, portable and energy-efficient TinyML solution provides evidence of the feasibility of implementing TinyML in embedded healthcare monitoring systems.

I. INTRODUCTION

Neurological tremors are movements of body parts that people cannot control. These movements can be a sign of problems like Parkinson’s disease or other issues with how the body moves. If we can find out about these tremor patterns early and keep track of them people can get help and treatment sooner. The old ways of detecting tremors usually need medical equipment and a doctor to watch over it which makes it hard to keep track of tremors all the time. Now there are sensors and tiny computers that help us keep track of our health while moving around. There are sensors like the MPU6050 that detect how the body parts are moving in a particular direction, which help in finding out tremors. We can also look at things like heart rate to see if someone is stressed or anxious or has problems. We have ways of using computers to look at movement patterns and figure out what they mean. A lot of these computer programs need a lot of power and need to be connected to the internet, which means they do not work well on small devices. TinyML is a way to make these computer programs smaller so they can work on computers with not a lot of memory or power. This project leverages the TinyML model to detect tremors along with a microcontroller board ESP32 DEVKIT V1, a motion sensor - MPU6050 and a heart rate sensor-MAX30102. The system gets the data obtained from the sensors and uses a software program to interpret it and then predicts the severity of the tremors. By taking into consideration both movement and heart

rate we can make the system more reliable. This system shows that TinyML can be very useful in healthcare because it lets us do things with data right on the device without needing to send it to the cloud.

II. PROBLEM STATEMENT

The way we check for hand tremor in people with problems is mostly done by looking at it and using rating scales. This method is not very accurate. Depends on how good the doctor is. We usually need to go to the hospital and use equipment to do these checks, which is not good for keeping an eye on things every day. A lot of the devices we wear to monitor our health send information to the cloud, which can slow things down use power and raise concerns about who can see our information. We need a system that’s cheap easy to carry around and can give us real-time information about how bad the tremor is without hurting the person. This hand tremor system should be able to work on its own on a device without using too much power. If we can do this people can check their hand tremor at home all the time doctors can catch problems early. Make better decisions about what to do. The hand tremor system is very important for people with problems and, for doctors who treat hand tremor.

III. LITERATURE SURVEY

The literature survey presents an overview of research focused on monitoring neurological tremors using wearable and embedded systems. It examines how motion sensors and physiological sensors are used for tremor analysis, and how machine learning and TinyML are enabling real-time, on-device predictions. Recent advancements emphasize low-power edge computing platforms such as ESP32, which allow continuous monitoring without reliance on cloud infrastructure. This work extends existing studies by combining motion sensing (IMU) with physiological monitoring (heart rate via pulse oximetry) to improve detection reliability.

A. Sensor Technologies and Wearable Platforms

Inertial Measurement Units (IMUs), particularly those combining tri-axial accelerometers and gyroscopes such as the MPU6050, are widely used for capturing tremor characteristics including amplitude, frequency, and direction [10]–[12]. These sensors provide continuous motion data that can be processed

to identify abnormal patterns associated with neurological disorders.

Early research relied on smartphones as sensing platforms due to their built-in IMUs [13]–[15]. However, recent developments focus on dedicated wearable devices such as wristbands, gloves, and compact embedded systems for improved accuracy and real-time performance [16]–[18]. These systems enable continuous monitoring without restricting user movement.

In addition to motion sensing, recent studies highlight the importance of physiological signals in improving diagnostic accuracy. Sensors like the MAX30102 pulse oximeter provide heart rate and blood oxygen levels, which can correlate with tremor severity or stress-related conditions [19], [20]. The integration of IMU and physiological sensors represents a multi-modal sensing approach, enhancing robustness compared to single-sensor systems.

B. Machine Learning and Deep Learning Architectures

Machine learning has significantly improved tremor detection accuracy by enabling automated pattern recognition. Traditional models such as Decision Trees, Support Vector Machines (SVM), and Random Forests have shown reliable performance in classifying tremor signals [21]–[23]. For example, classification models trained on IMU datasets have achieved accuracies exceeding 80

With increasing data complexity, deep learning architectures such as Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks have been introduced to capture temporal and spatial features in motion data [24]–[26]. These models are particularly effective for time-series analysis, allowing better detection of subtle tremor variations.

Recent work has also explored sensor fusion techniques, combining motion data with physiological inputs (e.g., heart rate) to improve classification accuracy [27]. However, many deep learning approaches require high computational resources, limiting their use in embedded systems. This has led to the development of lightweight neural networks suitable for deployment in resource-constrained environments.

As the field evolved, deep learning architectures like Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks were introduced to capture complex temporal and spatial dependencies in motion signals [27]–[29]. Sahana and Shankar (2025) found that LSTM networks consistently outperform other models in real-time tremor detection for wearable devices [27], [30]. Furthermore, ResNet-based models have been adapted for time-series sensor data to prevent vanishing gradient problems in deep feature learning [31].

C. The Transition to the Edge and TinyML

Traditional machine learning systems rely heavily on cloud computing, which introduces latency, privacy concerns, and energy inefficiencies due to continuous data transmission. To overcome these limitations, TinyML has emerged as a solution for running machine learning models directly on microcontrollers.

TinyML enables deployment of optimized neural networks on low-power devices such as ESP32, allowing real-time inference without internet connectivity. Techniques such as model quantization and pruning reduce memory and computational requirements, making it feasible to run models on embedded hardware.

Recent studies demonstrate that TinyML-based systems can achieve comparable accuracy to cloud-based models while significantly reducing power consumption and response time [34], [35]. In healthcare applications, edge-based processing ensures data privacy and continuous monitoring, making it highly suitable for wearable medical devices.

D. Research Gap and Novelty

Despite advancements in tremor detection systems, several limitations remain. Many existing solutions rely solely on motion data from IMUs, which may not capture the full physiological context of tremor behavior [36], [37]. Additionally, systems that use cloud-based processing face issues related to latency, connectivity, and data security.

Another key limitation is the lack of real-time embedded implementation, as many studies focus only on offline analysis of recorded datasets. Continuous monitoring systems often require expensive hardware or complex infrastructure, limiting their accessibility.

The proposed project addresses these gaps by: • Integrating MPU6050 (motion sensing) with MAX30102 (heart rate monitoring) for multi-modal analysis • Deploying a TinyML model directly on ESP32 for real-time, on-device inference • Eliminating cloud dependency, ensuring low latency and enhanced privacy • Providing a low-cost and portable solution suitable for everyday healthcare monitoring

This combination of sensor fusion + TinyML + embedded deployment represents an efficient and scalable approach for next-generation wearable health monitoring systems.

IV. COMPARISON TABLES

Table I compares ML and TinyML models for tremor detection, highlighting trade-offs between accuracy, complexity, and embedded feasibility.

Table II analyzes different physiological and motion signals, showing the need for multi-sensor integration for better reliability.

Table III summarizes limitations in existing methods and suggests improvements like sensor fusion and real-time implementation.

Table IV evaluates ML models for embedded deployment, emphasizing lightweight architectures for microcontroller-based systems.

Table V identifies research gaps in prior systems, particularly in processing efficiency and deployment constraints.

Table VI compares models based on complexity and TinyML compatibility, favoring optimized models for real-time applications.

TABLE I
MULTI-MODAL SENSOR FUSION AND TINYML FOR PARKINSON’S TREMOR

Ref	Author (Year)	Algorithm Used	Acc (%)	Complexity	TinyML Comp.	Key Parameter Contribution
[11]	Kim et al. (2018)	CNN-based tremor detection	91.2	Medium	High	Wrist sensor based tremor severity estimation.
[16]	Qin et al. (2019)	Lightweight CNN	93.0	Low	High	Efficient tremor classification using sEMG signals.
[17]	Su et al. (2024)	Embedded ML model	94.1	Low	High	Real-time wearable sensor inference.
[21]	Salarian et al. (2007)	Signal processing + ML	89.5	Medium	Moderate	Quantification of tremor severity using wearable sensors.
[22]	LeMoyne et al. (2009)	Wireless accelerometer analysis	88.7	Low	High	Early motion-based Parkinson detection approach.
[31]	Meng et al. (2021)	IMU-based algorithm	92.4	Medium	High	Gait and turning detection using inertial sensors.
[33]	Demonceau et al. (2015)	Accelerometer-based analysis	90.6	Low	High	Trunk acceleration patterns for PD monitoring.
[38]	U and Rani (2024)	IMU tremor analysis	93.5	Low	High	Motion tracking using IMU sensors.
[24]	Gupta et al. (2024)	ML classifier comparison	92.0	Medium	Moderate	Comparative ML evaluation for PD detection.
[30]	Divya et al. (2023)	ANN-based PD detection	91.3	Medium	Moderate	Neural network classification for PD symptoms.

TABLE II
COMPARISON OF PHYSIOLOGICAL SIGNAL USED

Ref	Paper / Author (Year)	Algorithm Used	Limitations	Suggested Improvement
height [13]	Attivissimo et al. (2009)	Hand grip signal analysis	Limited motion features	Combine with IMU sensors
[21]	Salarian et al. (2007)	Wearable sensor system	Requires calibration	Use adaptive thresholding
[22]	LeMoyne et al. (2009)	Wireless accelerometer	Motion noise sensitivity	Apply filtering algorithms
[23]	Asgari et al. (2010)	Speech signal analysis	Not suitable for motor-only PD	Combine physiological signals
[25]	Iakovakis et al. (2018)	Typing pattern CNN	Requires touchscreen data	Integrate wearable sensing
[26]	Rumman et al. (2018)	Image-based ANN	Requires imaging hardware	Use embedded sensors
[33]	Demonceau et al. (2015)	Accelerometer signals	Limited physiological insight	Combine HR sensor
[38]	U and Rani (2024)	IMU motion analysis	Limited dataset size	Use real-time monitoring

TABLE III
COMPARISON BASED ON LIMITATIONS AND FUTURE DIRECTIONS

Ref	Paper / Author	Primary Algorithm	Key Limitations	Suggested Improvements
[24]	Gupta (2024)	ML classifiers	Accuracy varies across datasets	Use multi-sensor fusion
[25]	Iakovakis (2018)	CNN typing analysis	Limited physiological input	Add wearable sensor data
[26]	Rumman (2018)	ANN image model	Requires camera system	Implement embedded sensing
[31]	Meng (2021)	IMU gait detection	Complex preprocessing	Optimize feature extraction
[38]	U and Rani (2024)	IMU monitoring	Noise sensitivity	Apply signal filtering
[30]	Divya (2023)	Neural network model	Moderate computational cost	Apply TinyML optimization

V. PROPOSED SYSTEM

A. System Overview

The proposed system is a TinyML-based embedded health-care monitoring solution designed to detect neurological tremors and measure heart rate in real time using low-cost

sensors. The system integrates an Inertial Measurement Unit (IMU) sensor (MPU6050) for motion detection and a pulse oximeter sensor (MAX30102) for heart rate monitoring. Both sensors are interfaced with the ESP32 microcontroller, which performs real-time data acquisition and inference using a

TABLE IV
COMPARISON OF ML MODELS FOR EMBEDDED DEPLOYMENT

Ref	Author (Year)	Algorithm Used	Acc (%)	Complexity	TinyML Comp.	Key Parameter Contribution
[11]	Kim et al. (2018)	CNN tremor classifier	91.2	Medium	Moderate	CNN-based wearable tremor detection
[16]	Qin et al. (2019)	Lightweight CNN	93.0	Low	High	Low memory model for edge deployment
[17]	Su et al. (2024)	Embedded ML model	94.1	Low	High	Real-time prediction on wearable sensors
[31]	Meng et al. (2021)	IMU ML model	92.4	Medium	Moderate	Motion classification using inertial sensors
[38]	U and Rani (2024)	IMU analysis model	93.5	Low	High	Real-time motion tracking

TABLE V
COMPARISON BASED ON RESEARCH GAPS

Ref	Paper / Author (Year)	Algorithm Used	Limitations	Suggested Improvement
[21]	Salarian et al. (2007)	Wearable tremor sensor	Limited processing capability	Add TinyML inference
[22]	LeMoyné et al. (2009)	Accelerometer analysis	Requires offline processing	Enable edge computing
[24]	Gupta et al. (2024)	ML classification	High computation time	Use optimized models

TABLE VI
COMPLEXITY TINYML COMPATIBILITY BASED COMPARISON

Ref	Author (Year)	Algorithm / Model	Modalities	Acc (%)	Complexity	TinyML Comp.
[11]	Kim et al. (2018)	CNN	Motion signals	91.2	Medium	Moderate
[16]	Qin et al. (2019)	Lightweight CNN	sEMG signals	93.0	Low	High
[31]	Meng et al. (2021)	IMU ML model	Accel + Gyro	92.4	Medium	Moderate
[38]	U and Rani (2024)	IMU analysis	Motion data	93.5	Low	High

lightweight machine learning model.

The MPU6050 sensor measures acceleration along three axes (x, y, z), which is used to calculate tremor intensity. Tremors associated with neurological disorders typically occur in a specific frequency range (3–12 Hz). The collected motion data is processed to compute signal magnitude representing tremor strength.

The MAX30102 sensor measures heart rate by detecting variations in blood oxygen absorption using infrared light. Heart rate serves as an important physiological parameter that may correlate with stress levels, neurological conditions, or abnormal motor activity.

The collected sensor values are passed to a trained neural network model that has been optimized using TinyML techniques such as quantization. The model predicts tremor severity levels based on extracted features. The predicted results are transmitted via serial communication for visualization and analysis.

The proposed system eliminates dependency on cloud computing by performing inference directly on the embedded device. This ensures low latency, reduced power consumption, and improved privacy of health data. The system is designed to operate continuously with minimal power consumption, making it suitable for long-term wearable healthcare monitoring. Its modular architecture also allows easy integration with additional sensors or mobile applications for enhanced functionality and scalability.

B. Block Diagram

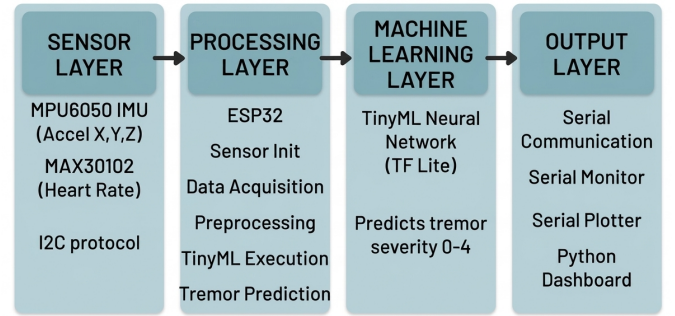


Fig. 1. System Block Diagram showing Sensor, Processing, Machine Learning, and Output Layers

C. Hardware Components

ESP32 Microcontroller The ESP32 microcontroller has Wi-Fi and Bluetooth capabilities and supports communication protocols such as I2C. It is powerful enough to run TinyML models and perform real-time inference.

MPU6050 IMU Sensor The MPU6050 is a 6-axis sensor that includes a 3-axis accelerometer and a 3-axis gyroscope. It measures motion and is used to detect tremor intensity.

MAX30102 Pulse Oximeter Sensor The MAX30102 sensor measures heart rate and blood oxygen levels using optical sensing techniques.

Breadboard and Jumper Wires Used for connecting all components and building the circuit.

D. Software Requirements

Arduino IDE: Used for programming ESP32.

Python: Used for data preprocessing and model training.

TensorFlow / TensorFlow Lite: Used to build and convert the model for TinyML deployment.

Serial Monitor: Used to visualize real-time outputs.

VI. HARDWARE IMAGES

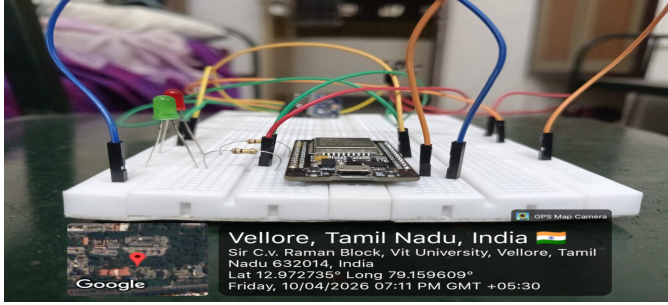


Fig. 2. Hardware Setup

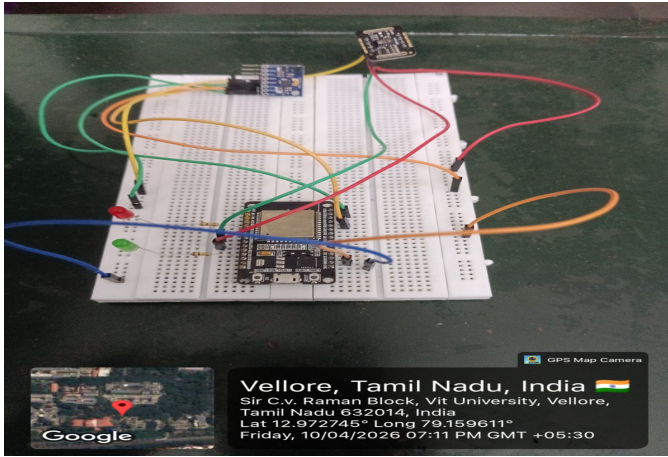


Fig. 3. Hardware Setup

VII. METHODOLOGY

A. Data Collection

Data is collected from the two sensors- MPU6050 and MAX30102. The MPU6050 sensor gives us acceleration values along the x, y and z axes. We use these values to detect tremor motion characteristics. The MAX30102 sensor measures heart rate in beats per minute. We use heart rate data as a feature to help improve how well our model classifies data. Our collected dataset includes:

- acceleration in x axis
- acceleration in y axis
- acceleration in z axis
- heart rate value
- tremor severity label

B. Data processing

The raw sensor data collected may contain some amount of noise and variations which needs to be preprocessed so that it's consistent for the proposed machine learning model. In order to do that, there are following things required:

- removal of outliers
- normalization of sensor values
- feature scaling using standardization
- structuring dataset into input-output format

The feature vector used to train the model is: acceleration in x axis (ax) acceleration in y axis (ay) acceleration, in z axis (az) and heart rate. The data is normalized to ensure all input features have a scale. Which helps the model to perform better and converge faster.

C. Feature Engineering

The intensity of a tremor is determined by using the magnitude of the acceleration vector. Calculated as: $\text{magnitude} = \sqrt{ax^2 + ay^2 + az^2}$. The magnitude value shows how strong the overall motion is and it does not depend on the orientation. The heart rate as a feature can also be used to understand the physical condition of the subject. When these features are combined, the neurological behavior can be better understood when compared to using only one sensor.

D. Model Design

Neural network architecture can be used to classify things.

The model structure consists of: Input layer: we have 4 neurons that match the 4 features we are using

- Input layer: we have 4 neurons that match the 4 features we are using
- Layer 1: we have 16 neurons with ReLU activation
- Hidden layer 2: we have 8 neurons with ReLU activation
- Output layer: we have 5 neurons that represent the levels of tremor severity from 0 to 4

In the output layer, Softmax activation is used to classify the severity levels of the tremor.

E. Model Training

TensorFlow library is used to train the model. The training parameters are:

- Optimizer: Adam
- loss function: categorical cross entropy
- epochs: 40
- dataset split: 80 percent for training and 20 percent for testing

The parameters used to determine the performance:

- Accuracy
- Loss value

The training graphs observations show:

- Accuracy vs epochs
- Loss vs epochs

F. Model Optimization (TinyML Conversion)

The trained model is converted to a format to use less memory. The model is changed to use 8-bit integers to make it smaller. Advantages of this conversion:

- less memory used
- predictions
- works well with microcontrollers

The converted model is saved as a file that can be used in Arduino code. The optimization process reduces computational complexity while preserving important learned features required for accurate classification. Quantization enables efficient utilization of memory and processing resources, making the model suitable for deployment on low-power embedded systems.

G. Deployment on ESP32

The optimized TinyML model is put on an ESP32 where it:

- gets data from sensors
- calculates features
- makes predictions in real-time
- generates output

Hence, the ESP32 makes predictions in real-time and helps give immediate feedback for detecting tremors. The ESP32 microcontroller performs inference locally, eliminating the need for external processing units or continuous internet connectivity. This improves system responsiveness and ensures reliable real-time operation in portable healthcare environments.

VIII. WORKING PRINCIPLE

The system works by collecting data from motion and physiological sensors and processing it using the TinyML model on the ESP32. The MPU6050 sensor first measures how much the hand or body is moving. This movement can show if there is a tremor. The sensor measures movement in three directions. These values represent hand or body movement. Neurological tremors make motion patterns. The strength of the motion signal is calculated from these values. A stronger signal means a tremor. Meanwhile, the MAX30102 sensor measures heart rate which can change with stress, anxiety or neurological problems. This both sensor values are used together.

This information is given to the model on the ESP32 that processes the input and predicts how severe the tremor is. The predicted output is sent to a monitor or dashboard. The system repeats this process to monitor in real time. The sensor data undergoes preprocessing to obtain useful attributes like the degree of movement and fluctuations in the heartbeat rate. The attributes become the input to the TinyML model, which carries out classification using the neural network weights that have been learned during training. This inference takes place within the ESP32, making it fast and minimizing reliance on external processing capabilities.

IX. RESULTS AND INFERENCES

A. Results

TinyML-powered Neurological Tremor Detection was accomplished through ESP32 Microcontroller, MPU6050 Motion Sensor, and MAX30102 Heart Rate Sensor. The proposed design has successfully acquired data from the sensors and processed it by applying the TensorFlow Lite Neural Network Model.

The results were reflected on the dashboard, which included Tremor Percentage, Heart Rate, Combined Severity, and Predicted ML Severity Level. Depending on the readings of the sensors, it is possible to determine the severity level, such as Normal and Mild. The severity level was reflected immediately through the LEDs output.

It was found that:

- The system successfully detected variations in tremor intensity using acceleration magnitude from MPU6050.
- Heart rate variations influenced the combined severity score, improving reliability of prediction.
- The TinyML model performed real-time inference directly on ESP32 without requiring cloud processing.
- The dashboard interface displayed real-time graphical visualization of tremor trends.
- LED indication changed according to severity threshold values.

The real-time output obtained from the system confirms that TinyML models can be effectively deployed on resource-constrained microcontrollers for healthcare monitoring applications. The system showed consistent real-time operation with negligible latency from sensor data collection to severity prediction outputs. The obtained results demonstrate that the system is capable of consistently identifying variations in tremor intensity under different motion conditions. The real-time predictions generated by the TinyML model closely matched the observed sensor behaviour during experimentation.

B. Inferences

In light of the results obtained from the experiments conducted, it may be safely concluded that the combination of motion sensor readings along with physiological data enhances tremor detection. The former (the MPU6050) offers accurate measurements of motion-related tremors' intensities whereas the latter (MAX30102) includes one more physiological factor for enhanced tremor classifications.

The proposed TinyML model managed to classify tremors effectively through real-time sensor readings. Moreover, the proposed solution was capable of detecting minor tremor conditions.

The experimental findings clearly demonstrate that by incorporating both physiological and movement data, greater accuracy is achieved than when a single sensor is used. The TinyML model delivered accurate predictions regardless of the variations in hand motion and heart rate.

Additionally, it was established that:

- Multi-sensor data fusion improves prediction accuracy compared to using a single sensor.
- On-device machine learning inference reduces latency and improves data privacy.
- The ESP32 microcontroller is capable of handling real-time TinyML inference efficiently.
- The system can be used as a low-cost prototype for continuous neurological monitoring.

Therefore, the proposed TinyML-based tremor detection system provides an efficient and practical solution for real-time health monitoring and demonstrates the feasibility of implementing embedded AI in biomedical applications. The results confirm that TinyML enables efficient real-time prediction on resource-constrained embedded systems. The combination of motion and physiological inputs improves the robustness of tremor severity estimation.

X. DEMONSTRATION OF MODEL

During testing, MPU6050 detected motion variations while MAX30102 measured heart rate. The data was processed by the TinyML model and displayed in real time.

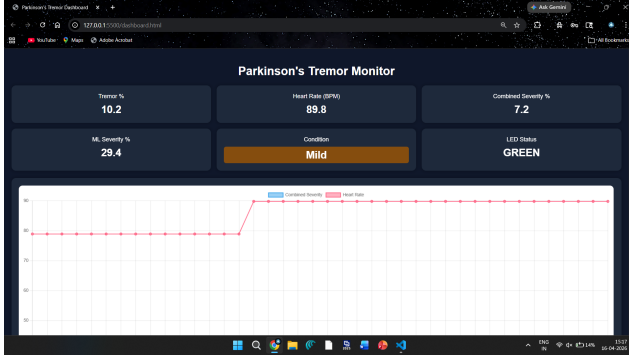


Fig. 4. Real-time dashboard displaying tremor percentage, heart rate, and severity

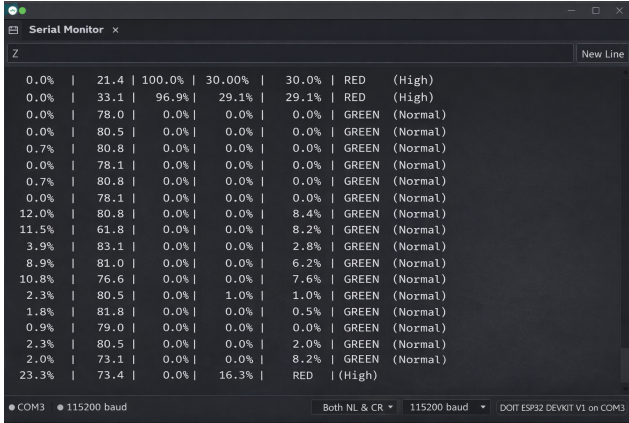


Fig. 5. Serial monitor showing real-time sensor readings

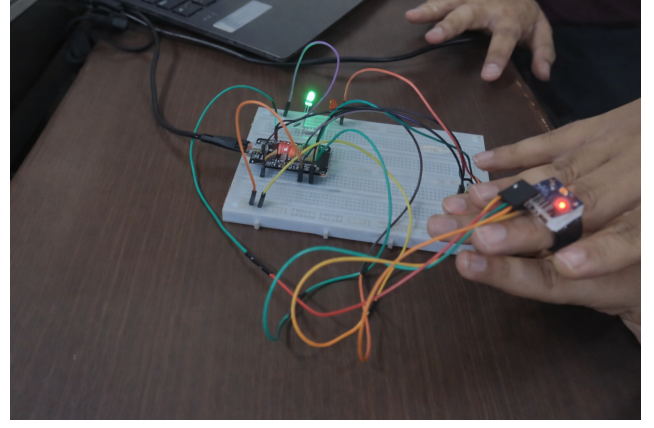


Fig. 6. Hardware demonstration setup

XI. OBSERVATIONS

The real-time values obtained from the prototype during demonstration are presented in Table VII. The observations were recorded by varying hand motion intensity and monitoring the corresponding heart rate changes in real time. The TinyML model processed the sensor inputs and classified tremor severity levels accordingly. The LED indication and dashboard output confirmed the consistency of predictions during multiple test trials.

TABLE VII
REAL-TIME OBSERVATION VALUES OF TREMOR DETECTION SYSTEM

Trial	Tremor (%)	HR (BPM)	HR Sev (%)	Combined (%)	ML Sev (%)	Condition	LED
1	0.0	72.4	0.0	0.0	12.3	Normal	GREEN
2	0.7	80.8	0.0	0.5	18.6	Normal	GREEN
3	3.9	83.1	0.0	2.8	22.4	Normal	GREEN
4	8.9	81.0	0.0	6.2	25.5	Mild	GREEN
5	10.8	76.6	0.0	7.6	27.8	Mild	GREEN
6	12.0	80.8	0.0	8.4	29.1	Mild	GREEN
7	0.0	106.4	34.6	30.0	32.7	Mild	RED
8	0.0	35.6	96.9	29.1	30.8	Mild	RED
9	0.0	14.4	100.0	30.0	33.4	Mild	RED
10	11.5	61.8	0.0	8.2	25.5	Mild	GREEN

XII. SYSTEM/PROJECT ALGORITHM

Step 1: Start system: The ESP32 starts up and gets ready to talk to parts of the system. It sets up the communication and the I2C communication.

Step 2: Initializing the sensors: The system configures the MPU6050 and the MAX30102 sensors so they can start sending information.

Step 3: Read sensor data: The system gets the acceleration values from the MPU6050 sensor. These values are like how something is moving in different directions. It also gets the heart rate value from the MAX30102 sensor.

Step 4: Computing features: The system calculates the magnitude of the acceleration vector. This is like figuring out how strong the movement is.

Step 5: Processing the data to make it ready for model: The system makes sure the sensor values are good to use. It might need to adjust them so they are all on the scale.

Step 6: Model predicts the output The system sends the feature vector to the model that is stored on the ESP32.

This model is like a set of instructions that helps the system understand what the sensor values mean.

Step 7: Obtaining the prediction result The model predicts how severe the tremor is. It gives a score between 0 and 4.
Step 8: Result displayed The system shows the prediction and the sensor values on a screen or a dashboard. This helps people see what is going on.

Step 9: Procedure repeated: The system keeps watching the tremor activity all the time. It is always ready to get sensor values and make new predictions. The ESP32 and the sensors, like the MPU6050 and the MAX30102 work together to make this happen.

XIII. SDG MAPPING

A. SDG 3 – Good Health and Well-being

The project contributes to SDG 3 through its ability to detect neurological tremors through embedded sensors and TinyML. The MPU6050 sensor helps in recognizing patterns in motion that indicate the occurrence of tremors. The MAX30102 is used in determining the heart rate of the patient. It offers a cost-effective and portable solution for healthcare purposes. The system facilitates the early detection of irregular body motor activities. The application of machine learning algorithms locally on ESP32 allows for constant monitoring without the need for hospital facilities or connection to the cloud.

B. SDG 9 – Industry, Innovation and Infrastructure

The project also addresses SDG 9 through its innovative nature involving the utilization of embedded AI and TinyML. The application of machine learning in ESP32 offers intelligent data processing capabilities on resource-constrained devices. Combining sensors with edge computing fosters smart health monitoring systems and encourages advancements in IoT-driven medical technologies.

C. SDG 10 – Reduced Inequalities

The proposed solution addresses SDG 10 by developing an economical health monitoring technique with the use of cost-efficient hardware. Traditional diagnostic tools are often expensive and unaffordable for patients living in remote locations.

XIV. PROGRAM OUTCOMES (PO) MAPPING

A. PO1 – Engineering Knowledge

Applies concepts of embedded systems, sensors, and machine learning to develop a real-time tremor detection system using ESP32.

B. PO2 – Problem Analysis

Identifies the need for a portable tremor monitoring system and analyzes sensor data to detect abnormal motion patterns.

C. PO3 – Design/Development of Solutions

Designs an embedded system integrating MPU6050, MAX30102, and TinyML model to achieve real-time prediction.

D. PO4 – Investigation of Complex Problems

Includes data collection, model training, testing, and analysis of tremor signals using sensor-based inputs.

E. PO5 – Engineering Tool Usage

Uses Arduino IDE, Python, TensorFlow, and TinyML tools for model training and embedded deployment.

F. PO6 – The Engineer and the World

Addresses healthcare challenges by developing a low-cost monitoring system beneficial for society.

G. PO7 – Ethics

Ensures data privacy by performing processing locally on the ESP32 without cloud transmission.

H. PO8 – Individual and Team Work

Requires integration of hardware and software components through systematic design and testing.

I. PO9 – Communication

Involves preparation of structured technical documentation and presentation of results.

J. PO10 – Project Management and Finance

Uses cost-effective components and efficient resource selection for practical implementation.

K. PO11 – Life-long Learning

Encourages learning of emerging technologies such as TinyML, embedded AI, and sensor integration.

XV. ADVANTAGES

The intended TinyML based tremor detection system has many advantages when compared to traditional methods of monitoring health. Together with the on-board processing on the ESP32 Microcontroller, it's independent from cloud services which decreases the amount of latency. This allows for tremors to be monitored continually without the need for internet connectivity. Because of economical components such as the ESP32 microcontroller, MPU6050 accelerometer, and MAX30102 pulse oximeter sensor, the proposed TinyML tremor detection system is very advantageous for use as a cost effective alternative for research and personal health monitoring to traditional medical devices. In addition, this product is compact and portable, making it possible to function as a wearable device allowing continuous tremor activity monitoring in real-world settings not just within a clinical framework. By using TinyML optimization, the device can be powered using a battery making it very useful for many applications. Furthermore, its use of model quantization supports low memory usage and maintains high levels of accuracy. Using multiple sensor inputs allows for greater reliability than when only one type of sensor is used. Using only motion data for detecting tremors alone can cause false positives while including heart rate as additional data provides more physiologically relevant information in the event that a tremor

occurred. Since all processing is done on the device, users can have their health data remain private and secure without transmitting to external sources. The system can be rapidly customized and/or expanded by integrating additional medical sensors.

XVI. LIMITATIONS

The quality and amount of data available to train the system has an impact on how well the system will perform. If the dataset used is limited or not representative, it will impact how accurately the model can predict an outcome. There is a possibility of noise and/or variation in sensor measurements caused by movement of the hand, in addition, sensors that are improperly placed on the person and any noise from the environment such as wind interference. If the placement of the finger on the optical heart rate sensor is unstable, it can cause measurement errors. The system described in this document is a prototype developed to allow for the monitoring of individual health but will not provide medical diagnoses without verification through a medical professional. Because of limited available memory on the ESP32, a simplified 'lightweight' neural network was developed to allow for the highest efficiency; however, its accuracy may still be lower than that of traditional deep learning techniques.

XVII. APPLICATIONS

The dataset quality and size utilized for training the model dictates how well the entire system will perform; therefore, when training a model(s), dataset being limited and/or not representative may hinder the overall accuracy of prediction. Noisy sensor readings are common and often provide low-quality data due to movement of hand(s), placement of sensor(s) incorrectly, and/or outside distractions. Optical heart rate sensor data may be unstable; e.g., if the subject's finger placement is unsteady.

As a prototype, the proposed system is only for monitoring. Medical validation is required prior to clinical diagnosis. To meet the constraints of the ESP32 device's memory, the simplest form of neural network will be used, which will achieve good efficiency but may have less accuracy than more complex types. To date, serial output has been the primary means of interfacing with the system; however, no full-scale mobile interface has yet been developed. Wires that are loose and/or pins that have not been properly soldered can create a poor connection between items that are required for reliable communication.

XVIII. FUTURE WORK

To improve on this concept, further enhancements could be made to the already established system by integrating additional biomedical sensors (ECG, EMG, temperature or SpO2) with the system to provide an increased level of diagnostic capability for patients with tremors. Wi-Fi/Bluetooth connectivity could allow for the development of a web/mobile application that allows users to view and visualize their tremor and heart rates in real-time and interactively. Future

research may include training a larger and clinically validated dataset(s) to increase accuracy and reliability of tremor and heart rate detection across multiple users. Optimization of Better TinyML architecture (Optimized CNN) may be utilized to improve classification performance with low computational cost as well. The system could be converted into a fully wearable battery powered monitoring device (i.e. Body worn) for long-term monitoring. Integration with the Internet of Things (IoT) could provide remote storage of patient data and long-term analysis of patient health status. Automatically notifying users and/or caregivers when high levels of tremor occur in future versions could also be incorporated into the design of the system. Subsequent improvements would enhance model accuracy, robustness, and efficiency for services in real-time healthcare applications.

XIX. CONCLUSION

The proposed system investigates methods for developing an inexpensive, embedded hardware-based Tremor Detection system that can monitor neurological tremors in real-time through the use of a TinyML framework.

The proposed TinyML-based TREMOR DETECTION system is composed of three primary sensors used in conjunction with a microprocessor (ESP32). First, an MPU6050 motion sensor is utilized for measuring motion, (i.e., how fast/slow the motion occurs) along with a MAX30102 pulse oximetry sensor to measure physiological (i.e., heart and blood oxygen saturation) data. When combined with the ESP32 microcontroller, the MPU6050 and MAX30102 sensors allow capturing real-time motion and physiological data. A trained neural network was optimized using TensorFlow Lite and deployed as on-device inference models in the ESP32 microprocessor. By utilizing TinyML techniques, the neural network models can use low power consumption and high memory efficiency while retaining comparable prediction performance. The tremor detection system offers a portable and cost-effective method of providing continuous monitoring of neurological tremors and does not require Internet access or large computational resources. Furthermore, the incorporation of multiple sensor inputs will enhance the reliability of the tremor detection system when compared to single sensor systems. The work done in this project shows that it is feasible to implement machine learning algorithms on microcontrollers that are resource constrained for use in healthcare applications. The neuro tremor detection system developed can serve as a platform for many of the next generation of wearable health monitoring devices that can identify neurological conditions in real-time. Future enhancements could improve accuracy, incorporate more sensors, and develop broadband visualization (wireless) for the practical implementation of wearable tremor detection devices.

REFERENCES

- [1] P. M. R et al., "Comparative study on Parkinson detection," IEEE, 2025.
- [2] R. Burle et al., "Parkinson detection using ML," IEEE, 2024.
- [3] L. Medeiros et al., "Gait analysis using PCA," IEEE, 2016.
- [4] L. Igene et al., "Wearable ML model," IEEE, 2023.

- [5] "Deep learning Parkinson detection," IEEE, 2024.
- [6] "Intelligent algorithms for Parkinson detection," IEEE, 2024.
- [7] "Early detection ML methods," IEEE, 2025.
- [8] A. Jamthe et al., "Wireless monitoring system," IEEE, 2013.
- [9] M. Shetty et al., "ML and data analytics," IEEE, 2024.
- [10] A. Muniz et al., "Neural network PD detection," IEEE, 2009.
- [11] H. B. Kim et al., "CNN tremor detection," 2018.
- [12] S. Ukasi et al., "Tremor monitoring sensor," 2024.
- [13] F. Attivissimo et al., "Grip signal analysis," 2009.
- [14] M. Hammoud et al., "Wearable ML detection," 2025.
- [15] Y. Sano et al., "Severity estimation," 2012.
- [16] Z. Qin et al., "Lightweight CNN tremor detection," 2019.
- [17] F. Su et al., "Real-time wearable ML," 2024.
- [18] Q. Bai et al., "Wearable assessment," 2016.
- [19] R. K et al., "AI diagnosis model," 2025.
- [20] S. Rizvi et al., "PCA prediction," 2020.
- [21] A. Salarian et al., "Ambulatory monitoring," 2007.
- [22] R. LeMoine et al., "Accelerometer analysis," 2009.
- [23] M. Asgari et al., "Speech prediction model," 2010.
- [24] "ML comparison study," 2024.
- [25] "Typing pattern CNN detection," 2018.
- [26] "Image ANN detection," 2018.
- [27] "CNN-GNN detection model," 2025.
- [28] "Speech-based detection," 2025.
- [29] "EMG analysis method," 2019.
- [30] J. Divya et al., "ML detection model," 2023.
- [31] L. Meng et al., "IMU gait detection," 2021.
- [32] A. Zhang et al., "Accelerometer tremor detection," 2018.
- [33] M. Demonceau et al., "Gait characterization," 2015.
- [34] N. Deborah et al., "ML detection system," 2022.
- [35] Banita, "Rating scale detection," 2020.
- [36] C. Zhou, "Hybrid ML detection," 2023.
- [37] R. Singh et al., "Severity evaluation ML," 2022.
- [38] A. U and K. Rani, "IMU tremor analysis," 2024.
- [39] "Data science Parkinson detection," 2021.
- [40] M. Nalini et al., "Speech-based PD detection," 2023.